

赛题三：PagedAttention 算子

1. 赛题背景

PagedAttention 通过将 KV Cache 拆分为固定大小的 block，并用块索引表实现非连续访存，在大模型推理中提升显存利用率并支持块共享。选手需要在前向阶段实现一个简化版 Query PagedAttention，重点在正确处理块索引和变长上下文，保证数值正确。

本赛题要求选手在 **Ascend 910B3** 硬件平台上，使用 **Triton-Ascend** 或 **TileLang-Ascend** 编写 **PagedAttention** 算子。

2. 算子功能

完成当前时刻的 query 与其上下文 KV 的注意力计算。规则：

- (1) 依据 `block_tables` 从非连续物理块中收集上下文 KV；
- (2) 支持不同序列拥有不同的 `context_lens`；
- (3) 允许多个序列共享同一物理块；
- (4) 仅需前向，不需反向或写缓存。

3. 输入输出

Query 形状为 `query: [num_seqs, num_heads, head_size]`；

Key/Value Cache 形状为

`key_cache: [num_blocks, num_heads, head_size, block_size]`，`value_cache` 同形；

Block_tables `block_tables: [num_seqs, max_num_blocks]`，`block_tables[t, i]` 给出第 `t` 个序列第 `i` 个逻辑块对应的物理块编号；

Context_lens `context_lens: [num_seqs]`，实际访问 token 总数为 `context_lens[t]`；

Out `output: [num_seqs, num_heads, head_size]`，在允许的浮点误差内需与参考实现一致。

4. 答题要求

代码框架在平台的共享文件存储中已经给出（权限为只读），路径如下。

源路径：[/pagedattention](#)

挂载路径：[/3-pagedattention/pagedattention](#) 

请你在已给出的 PagedAttention_triton.py 和 PagedAttention_tilelang.py 中任选其一，补全其中的 **todo**（可参考 paged_attention_ref），并运行文件通过结果正确性测试。请在希冀平台的对应作业中提交你的**代码文件**和**运行结果截图**。

注：你可以适当的合理修改给出的框架中 **PagedAttention_kernel** 的内容。